CS 499 Milestone Four: Database Enhancement Narrative

Travlr Getaways | Enhancement Three: Databases

Student: Matthew Wood | Course: CS 499 | Date: 10/13/2026

Artifact Description

For this enhancement, I used Travlr Getaways, the MEAN stack travel application I developed in CS 465 and continued improving for my CS 499 capstone. The application stores trip information in MongoDB and uses Mongoose models and Express API endpoints to retrieve and manage that information. I selected this artifact because it gave me a working database application where I could improve how data is validated, protected, queried, and maintained.

Reason for Selecting the Artifact

The original application could store and retrieve trips, but the database layer had room for improvement. I wanted to make the data more consistent and prevent invalid information from entering the collection. I also wanted to demonstrate more than basic create, read, update, and delete operations. Adding aggregation, improving error handling, and making database initialization safer allowed me to show practical database design and application development skills.

Enhancement Process

I started with the Mongoose trip schema in app_api/models/travlr.js. I added validation for required text fields, trimmed string values, and applied length restrictions. Trip codes must follow an alphanumeric format and remain unique. I also changed the price field, perPerson, to use a numeric value with a positive minimum and validation that limits prices to two decimal places. These checks help keep trip records consistent and make invalid data easier to catch.

Next, I updated the trip controller in app_api/controllers/trips.js. The API now validates incoming trip information, converts prices to numbers, and returns more useful HTTP responses for validation failures and duplicate trip codes. A duplicate code returns HTTP 409 instead of being treated as a general server error. Invalid values return HTTP 400. I also added a protected analytics endpoint that uses a MongoDB aggregation pipeline to calculate the number of trips, average price, minimum price, maximum price, and total price. The route requires authentication and is registered before the parameterized trip-code route so the analytics path is handled correctly.

After completing the initial database enhancement, I identified another issue with API input validation. Some malformed requests could still cause errors when the application attempted to process unexpected data types. I updated the validation logic to check the request structure and confirm that text fields contain strings before processing them. I tested this correction by

submitting a numeric value for a trip name while authenticated. The API returned HTTP 400 with a validation message instead of encountering an unexpected error. This final adjustment improved the reliability of the request-handling process.

I improved the database connection configuration in app_api/models/db.js by using environment-based database settings, a connection timeout, and connection and shutdown handling. I revised the seed script so it validates trip data before connecting and uses an upsert with $setOnInsert. This makes repeated seed runs non-destructive: existing trip records are preserved instead of overwritten. Finally, I changed the sample trip prices in app_server/data/trips.json from strings to numbers so the sample data matches the schema.

Testing and Results

I tested the updated schema with valid and invalid prices. Valid examples included 899, 899.99, 1099.50, and 0.01; invalid examples included zero, a negative price, and a value with more than two decimal places. All seven price-validation checks produced the expected results. I also tested the API with a duplicate trip code. The request returned HTTP 409, and the original trip remained unchanged.

I tested database create, update, validation, and delete operations using a temporary trip record, then removed that record. The original six trip records were preserved. An authenticated request to the analytics endpoint returned a count of six, an average price of 949.17, a minimum of 100, a maximum of 1399, and a total of 5695. A request without authentication returned HTTP 401.

I also checked the unique index on the trip code. The database rejected a duplicate code, and an indexed lookup for BAH001 used an IXSCAN rather than a collection scan. Because the dataset is small, I consider this evidence that the index is being used, not proof of a measurable speed improvement. After restarting the application, GET /api/trips returned HTTP 200 with the six existing records. I reran the seed script and confirmed that it processed the three seed entries without inserting duplicates or overwriting existing records.

Challenges and Decisions

One important decision was to enforce data rules at more than one level. API validation provides clear feedback to clients, while Mongoose schema validation protects the stored records if data reaches the model through another path. I also had to keep prices consistent across the sample JSON, request handling, and database schema. For analytics, I chose aggregation in MongoDB rather than retrieving every trip and calculating the results in application code. This keeps the calculation close to the stored data and provides one endpoint for summary information.

Another concern was protecting existing data. I used a non-destructive seed process so rerunning the script would not erase edits made to trip records. I also replaced the JWT secret in the shared configuration example with a placeholder. These changes support a safer development workflow without requiring the user to share an actual secret.

Course Outcomes and Professional Growth

This enhancement demonstrates my ability to design and work with databases, apply validation rules, use database queries and aggregation, and integrate database behavior with a web API. It also shows how I approach reliability: I checked error cases, tested authentication, verified index use, and made sure the seed process preserved existing information.

Compared with the earlier version of Travlr Getaways, the enhanced artifact has stronger data integrity and more useful database functionality. This work helped me better understand how decisions in a schema affect API behavior and how testing needs to cover both successful operations and failures. These are skills I can apply to future computer science projects that store and process data.

Limitations and Future Improvements

The analytics endpoint currently provides summary statistics for the entire trips collection. A future version could support filtering by destination, price range, or travel dates and could add automated integration tests that run against a dedicated test database. I would also consider pagination and additional query indexes if the number of trip records grows. I did not treat the current six-record index check as a benchmark for production-scale performance.

Artifact Submission

The accompanying CS499 M5M4A.zip contains two folders: Original (the version before the database enhancement) and Enhanced (the completed database enhancement). The comparison shows the changes to the schema, controllers, routes, connection handling, seed script, configuration example, and sample trip data.